

Package ‘flowMagic’

May 28, 2026

Type Package

Title Automated gating of flow cytometry data

Version 0.99

Description tool to perform the automated gating of bivariate flow cytometry data.

License Apache License 2.0 | file LICENSE + file NOTICE

Encoding UTF-8

LazyData true

Roxygen list(markdown = TRUE)

Imports caret,
ggplot2,
stringr,
parallel,
randomForest,
concaveman,
sp,
sf,
stats,
grDevices,
doParallel,
nnet,
sm,
pracma,
patchwork,
ggExtra,
RColorBrewer,
dplyr

Suggests flowCore,
flowWorkspace,
Biobase,
data.tree,
visNetwork,
klaR(<= 0.6-12),
remotes,

CytoML,
 plotly,
 magrittr,
 geometry

Config/roxygen2/version 8.0.0

Contents

add_labels_column	3
assign_events_to_nearest_centroids	4
check_polygons_intersection	5
compute_gates	5
convert_to_integers_chr	6
csv_to_dens	6
exports_plots	7
export_raw_gs_plots	8
extract_polygon_gates	9
flowmagic_pred_to_gs	10
flowmagic_pred_to_poly_gates	10
format_channel_info	11
get_centroids	12
get_classes_expr_df	13
get_density_features	14
get_density_scores	14
get_distance_loc_vs_test	15
get_dist_template	15
get_flowframe_from_gs	16
get_gate_channels	17
get_hierarchy_all_pops	18
get_hull_all_gates	18
get_indices_cross_val	19
get_list_df_gated_plots	20
get_local_train_sets	20
get_pops_hierarchy_list	21
get_pop_multiclass	22
get_slot_hierarchy_list	22
get_test_sets	23
get_train_data	23
get_weights_density_features	24
import_gating_info	25
import_reference_csv	25
import_test_set_csv	26
import_test_set_fcs	26
magicGating	27
magicPlot	28
magicplot_3D	29
magicPlot_fs	30

Value

Dataframe.

Examples

```
add_labels_column()
```

```
assign_events_to_nearest_centroids
    assign_events_to_nearest_centroids
```

Description

function to assign events to class with nearest centroid.

Usage

```
assign_events_to_nearest_centroids(
    gated_df,
    n_cores = 1,
    method_dist = "euclidean",
    thr_dist = 0.15,
    include_zero = F,
    remove_centroids = T
)
```

Arguments

<code>gated_df</code>	dataframe with labels (third column).
<code>n_cores</code>	Number of cores. Default to 1.
<code>method_dist</code>	Distance method calculation. Default to euclidean.
<code>thr_dist</code>	Distance threshold for centroids calculation. Default to 0.15.
<code>include_zero</code>	Consider centroid of label 0. Default to False.
<code>remove_centroids</code>	Remove centroids too near each other based on <code>thr_dist</code> value.

Value

Dataframe.

Examples

```
assign_events_to_nearest_centroids()
```

check_polygons_intersection	
	<i>extract_polygon_gates</i>

Description

function to check polygons intersection.

Usage

check_polygons_intersection(list_df_hull)

Arguments

list_df_hull List of polygons coordinates

Value

float

Examples

check_polygons_intersection()

compute_gates	<i>compute_gates</i>
---------------	----------------------

Description

function to assign events based on polygon gates.

Usage

compute_gates(gated_df, list_final_polygons_coors, no_classes = F)

Arguments

- gated_df dataframe with labels (third column).
- list_final_polygons_coors List of dataframes containing polygon coordinates.
- no_classes Generate third column of labels. Default to False.

Value

Dataframe.

Examples

```
compute_gates()
```

```
convert_to_integers_chr
      convert_to_integers_chr
```

Description

function to text classes to integers-like classes if necessary.

Usage

```
convert_to_integers_chr(df)
```

Arguments

df Dataframe with classes (third column) to check and eventually modify

Value

Dataframe

Examples

```
convert_to_integers_chr()
```

```
csv_to_dens                      csv_to_dens
```

Description

function to get density of events (with classes associated if present).

Usage

```
csv_to_dens(df, with_classes = T, n_coord = "df", normalize_data = T)
```

Arguments

df Dataframe of marker expression values.
with_classes Consider classes. Default to True.
n_coord Grid size. Default to df.
normalize_data If True, data is normalized to 0-1 range. Default to True.

Value

Dataframe.

Examples

```
csv_to_dens()
```

exports_plots	<i>exports_plots</i>
---------------	----------------------

Description

function to generate plots (no hierarchy) from list of labelled dataframes.

Usage

```
exports_plots(  
  list_gated_data,  
  path_output,  
  n_cores = 1,  
  type_plot = "dens",  
  show_legend = T,  
  x_lab = "x",  
  y_lab = "y",  
  size_title_x = 23,  
  size_title_y = 23,  
  aspect_ratio = NULL,  
  w_val = 16,  
  h_val = 10,  
  size_axis_text = 25,  
  export_csv = F,  
  side_by_side = F,  
  ...  
)
```

Arguments

list_gated_data	list of dataframes. Each dataframe has 3 columns: marker 1 values, marker 2 values and label column.
path_output	Path to the directory where to export the plots.
n_cores	Number of cores to use. Default to 1.
type_plot	the user can choose between density (= "dens") or label assignment visualization (= "ML")
show_legend	If True it shows the legend for the label assignment visualization. Default to True.

<code>x_lab</code>	x-axis label.
<code>y_lab</code>	y-axis label.
<code>size_title_x</code>	Size x axis label.
<code>size_title_y</code>	Size y axis label.
<code>aspect_ratio</code>	Set aspect ratio. Default to NULL> If = 1, y and x axis ticks have same distance.
<code>w_val</code>	width value. Default to 16.
<code>h_val</code>	height value. Default to 10.
<code>size_axis_text</code>	Size of ticks labels.
<code>export_csv</code>	Export plot data as csv files. Default to False.
<code>side_by_side</code>	Arrange dens plot and ML plot side-by-side. Default to False.

Examples

```
exports_plots()
```

<code>export_raw_gs_plots</code>	<i>export_raw_gs_plots</i>
----------------------------------	----------------------------

Description

function to generate ungated plots from selected gs node

Usage

```
export_raw_gs_plots(
  gs,
  node_name,
  channel_x,
  channel_y,
  path_output,
  n_cores = 1,
  x_lab = NULL,
  y_lab = NULL,
  w_val = 7,
  h_val = 7,
  size_points = 1,
  return_data = F,
  ...
)
```


Arguments

gs	GatingSet
path_output	Path to the directory where to export the plots.
n_cores	Number of cores to use. Default to 1.
x_lab	x-axis label. Default to NULL (inherited from GatingSet metadata).
y_lab	y-axis label. Default to NULL (inherited from GatingSet metadata).
w_val	width value. Default to 7 inches.
h_val	height value. Default to 7 inches.
size_points	Size points scatter plot.
return_data	If TRUE, return the list of dataframes used to generate the plots. Default to FALSE.

Examples

```
export_raw_gs_plots()
```

```
extract_polygon_gates extract_polygon_gates
```

Description

function to extract the polygon gates objects based on the convex hull and classes.

Usage

```
extract_polygon_gates(gated_df, concavity_val = 1, ...)
```

Arguments

gated_df	dataframe with labels (third column).
concavity_val	Concavity of polygons. Default to 1.

Value

List of dataframes.

Examples

```
extract_polygon_gates()
```

```
flowmagic_pred_to_gs  flowmagic_pred_to_gs
```

Description

function to add list of polygon objects generated by flowmagic_pred_to_poly_gates function to GatingSet object.

Usage

```
flowmagic_pred_to_gs(list_poly_gates, gs, parent_node)
```

Arguments

list_poly_gates	List of polygonGate objects generated using the flowmagic_pred_to_poly_gates
gs	GatingSet to update with the flowMagic polygons.
parent_node	Parent population name of the new gate.

Value

GatingSet

Examples

```
flowmagic_pred_to_gs()
```

```
flowmagic_pred_to_poly_gates
flowmagic_pred_to_poly_gates
```

Description

function to label points of dataframe based on polygon coordinates

Usage

```
flowmagic_pred_to_poly_gates(
  list_df,
  pred_label,
  gate_label,
  n_cores = 1,
  concavity_val = 10
)
```

Arguments

list_df	List of dataframes with three columns: marker 1, marker 2, class. Can get input generated by the magicPred function.
pred_label	Select prediction label to extract polygon.
gate_label	Label for the selected polygon within the GatingSet framework.
n_cores	Number of cores to use. Default to 1.
concavity_val	Concavity value for drawing polygon. Default to 10.

Value

list

Examples

```
flowmagic_pred_to_poly_gates()
```

format_channel_info	<i>Format gate channel information</i>
---------------------	--

Description

This helper function takes channel names used by a gate and matches them to the metadata stored in a flowFrame parameter table.

Usage

```
format_channel_info(channels, df_metadata)
```

Arguments

channels	Character vector. Channel names used by a gate. For example, c("BV480-A", "BV650-A").
df_metadata	Data frame. FlowFrame parameter metadata, usually from ff@parameters@data. Must contain columns name and desc.

Details

The metadata table usually comes from:

```
ff@parameters@data
```

It should contain at least the columns name and desc.

For channels with a marker description, the function returns:

```
channel = marker
```

For channels without a marker description, such as scatter channels, the function returns only the channel name.

Value

A single character string containing formatted channel information, with one channel per line.

Examples

```
## Not run:
ff <- flowMagic::get_flowframe_from_gs(
  gs = gs,
  node_name = "root",
  sample_id = 1
)

df_metadata <- ff@parameters@data

format_channel_info(
  channels = c("BV480-A", "BV650-A"),
  df_metadata = df_metadata
)
# "BV480-A = CD19\nBV650-A = NK1.1"

## End(Not run)
```

get_centroids

get_centroids

Description

function to get centroids for each label

Usage

```
get_centroids(
  df,
  low_thr = 0.1,
  up_thr = 0.9,
  thr_dist = 0.15,
  include_zero = F,
  remove_centroids = T
)
```

Arguments

df	dataframe with labels (third column).
low_thr	Lower threshold for quantile calculation.
up_thr	Upper threshold for quantile calculation.
thr_dist	Distance threshold for centroids calculation. Default to 0.15.

`include_zero` Consider centroid of label 0. Default to False.
`remove_centroids` Remove centroids too near each other based on `thr_dist` value.

Value

Dataframe.

Examples

```
get_centroids()
```

<code>get_classes_expr_df</code>	<code>get_classes_expr_df</code>
----------------------------------	----------------------------------

Description

function to get classes of original expression df based on density df predictions.

Usage

```
get_classes_expr_df(dens_df, original_df)
```

Arguments

`dens_df` Dataframe of density values.
`original_df` Original dataframe.

Value

Dataframe.

Examples

```
get_classes_expr_df()
```

get_density_features	<i>get_density_features</i>
----------------------	-----------------------------

Description

function to get density features only given a bivarite density csv.

Usage

```
get_density_features(df_dens, min_height = 0.06)
```

Arguments

df_dens	Dataframe of density estimates for both markers.
min_height	Minimum height of the peaks to consider.

Value

Vector of numbers.

Examples

```
get_density_features()
```

get_density_scores	<i>get_density_scores</i>
--------------------	---------------------------

Description

function to get scores for distance template calculation.

Usage

```
get_density_scores(df_template, df_test, select_density_features = NULL)
```

Arguments

df_template	Dataframe of template markers expression values.
df_test	Dataframe of test markers expression values.
select_density_features	Select features to use. Default to NULL.

Value

Matrix of numbers.

Examples

```
get_density_scores()
```

```
get_distance_loc_vs_test
    get_distance_loc_vs_test
```

Description

function to compare the similiraty between current test data and local training set.

Usage

```
get_distance_loc_vs_test(test_df, loc_df, show_plot = "none", nboot = 50)
```

Arguments

test_df	Dataframe with bivariate marker expression.
loc_df	Dataframe with bivariate marker expression.
show_plot	show density comparison. Default to none.
nboot	Number of permutations. Default to 50.

Value

List of p-values.

Examples

```
get_distance_loc_vs_test()
```

```
get_dist_template    get_dist_template
```

Description

function to get distance between template and test data.

Usage

```
get_dist_template(matrix_scores, dist_method = "euclidean")
```

Arguments

matrix_scores	Matrix of density features generated by get_density_scores function.
dist_method	Type of distance method calculation.

Value

Number.

Examples

```
get_dist_template()
```

```
get_flowframe_from_gs  get_flowframe_from_gs
```

Description

Extract a population from a GatingSet or GatingHierarchy as a flowFrame.

Usage

```
get_flowframe_from_gs(gs, node_name, sample_id = NULL)
```

Arguments

gs	A GatingSet or GatingHierarchy object. If a GatingSet is provided, sample_id must also be provided. If a GatingHierarchy is provided, sample_id is ignored.
node_name	Character. Name of the population node to extract, such as "root", "Live_cells", "Singlets", or another node present in the gating hierarchy.
sample_id	Character or NULL. Sample name to extract when gs is a GatingSet. The value must be present in sampleNames(gs). Default is NULL, which is allowed only when gs is already a GatingHierarchy.

Details

This function extracts the events belonging to a selected population node from either a GatingSet or a GatingHierarchy. The extracted data are returned as a flowFrame, which can then be used by downstream flowMagic functions.

If gs is a GatingSet, the user must provide sample_id because a GatingSet contains multiple samples. If gs is already a GatingHierarchy, sample_id is not needed because a GatingHierarchy represents one sample.

The function uses inherits() to check whether the input object is a GatingSet or a GatingHierarchy. This is preferred over direct class() comparisons because Bioconductor/S4 objects may have multiple classes or inherit from other classes.

Internally, the selected population is first extracted using flowWorkspace::gh_pop_get_data().

The resulting object is usually a cytoframe, which is then converted to a flowFrame using cytoframe_to_flowFrame().

Value

A flowFrame object containing the events from the selected population node.

Examples

```
# Extract from a GatingSet
ff <- get_flowframe_from_gs(
  gs = gs,
  node_name = "Live_cells",
  sample_id = "sample_01.fcs"
)

# Extract from a GatingHierarchy
gh <- gs[["sample_01.fcs"]]

ff <- get_flowframe_from_gs(
  gs = gh,
  node_name = "Live_cells"
)
```

get_gate_channels	<i>Extract gate channels for a population</i>
-------------------	---

Description

This helper function gets the gate used to define a population and extracts the flow cytometry channels used by that gate.

Usage

```
get_gate_channels(gs, pop)
```

Arguments

gs	A GatingSet or GatingHierarchy object.
pop	Character. Name of the population/node.

Details

In some cases, `flowWorkspace::gs_pop_get_gate()` returns the gate directly. In other cases, it returns a list with one gate per sample. This function handles that by using the first gate in the list.

Value

A character vector containing the channel names used by the gate. If the population is "root", if no gate is found, or if the channels cannot be extracted, an empty character vector is returned.

Examples

```
## Not run:
get_gate_channels(gs[[1]], "B_cells")
# [1] "BV480-A" "BV650-A"

## End(Not run)
```

```
get_hierarchy_all_pops
```

```
get_hierarchy_all_pops
```

Description

function to plot hierarchy of all nodes/cell populations from a GatingHierarchy object.

Usage

```
get_hierarchy_all_pops(gh, export_visnet = F, path.output = "None")
```

Arguments

`gh` GatingHierarchy.
`export_visnet` If true, it export visnetwork object.
`path.output` Path to save visnetwork object. Default to None.

Value

List of Dataframes.

Examples

```
get_hierarchy_all_pops()
```

```
get_hull_all_gates
```

```
get_hull_all_gates
```

Description

function to get the convex hull of all gates.

Usage

```
get_hull_all_gates(gated_df, concavity_val = 1, spar_val = 0.7, smoothing = F)
```

Arguments

gated_df	dataframe with labels (third column).
concavity_val	Values of concavity. Default to 1.
spar_val	Value of spar. Default to 0.7.
smoothing	Apply smoothing of lines? Default to F.

Value

List of dataframes.

Examples

```
get_hull_all_gates()
```

```
get_indices_cross_val  get_indices_cross_val
```

Description

function to get indices for cross val.

Usage

```
get_indices_cross_val(
  df_train,
  n_cores = 1,
  train_inds = "plot_num",
  val_inds = "none",
  n_train_plots = 5,
  n_folds = 5,
  seed = 40,
  n_val_plots = 5
)
```

Arguments

df_train	Dataframe of training features generated by the get_train_data function.
n_cores	Number of cores to use. Default to 1.
train_inds	Type of method to extract training indices: plot_num,rand_set_num,rand_set_n_gates_info.
val_inds	Type of method to extract validation indices: plot_num,rand_set_num,rand_set_n_gates_info.
n_train_plots	Number of training plots to include in training data for each iteration.
n_folds	Number of training iterations.
seed	Seed to randomly extract data.
n_val_plots	Number of training plots to include in validation data for each iteration.

Value

List of integers.

Examples

```
get_indices_cross_val()
```

```
get_list_df_gated_plots
```

```
get_list_df_gated_plots
```

Description

function to get labelled dataframe based on selected node for each sample in gs.

Usage

```
get_list_df_gated_plots(gs, gate_name, label_pop = NULL)
```

Arguments

<code>gs</code>	GatingSet
<code>gate_name</code>	Name of the Gating tree node whose gating data needs to be extracted.
<code>label_pop</code>	Set label for selected gate.

Value

List.

Examples

```
get_list_df_gated_plots()
```

```
get_local_train_sets    get_local_train_sets
```

Description

Based on the hierarchy calculated using the `get_hierarchy` function, we generate the local training sets gated by the biologists.

Usage

```
get_local_train_sets(gh, hierarchical_tree, info_hierarchy)
```

Arguments

`gh` GatingHierarchy.
`hierarchical_tree` Dataframe of hierarchy information generated by the `get_hierarchy` function.
`info_hierarchy` List of hierarchy information generated by the `get_hierarchy` function.

Value

List of Dataframes.

Examples

```
get_local_train_sets()
```

```
get_pops_hierarchy_list  
                          get_pops_hierarchy_list
```

Description

function to get info from a list of hierarchical dataset (it reports all the pops for each level in a vector).

Usage

```
get_pops_hierarchy_list(hierarchical_list)
```

Arguments

`hierarchical_list` list of populations names for each level.

Value

Vector of characters.

Examples

```
get_pops_hierarchy_list()
```

get_pop_multiclass	<i>get_pop_multiclass</i>
--------------------	---------------------------

Description

function to get the children of the selected population, useful for multiclass classification. The output is only the pops with same dimensions

Usage

```
get_pop_multiclass(gh, pop)
```

Arguments

gh	GatingHierarchy.
pop	Name of population to get events assignments.

Value

Vector of characters.

Examples

```
get_pop_multiclass()
```

get_slot_hierarchy_list	<i>get_slot_hierarchy_list</i>
-------------------------	--------------------------------

Description

function to access a slot of results from the hierarchy list based on the pop selected.

Usage

```
get_slot_hierarchy_list(hierarchical_list, pop_selected)
```

Arguments

hierarchical_list	list of populations names for each level.
pop_selected	Population name.

Value

Element of a list.

Examples

```
get_slot_hierarchy_list()
```

get_test_sets	<i>get_test_sets</i>
---------------	----------------------

Description

Function to obtain the data ready to be gated (validation set or test set). It takes as input a flowSet and generates a list of dataframes with a ML structure for each ungated fcs file. Each dataframe is the expression matrix of the root.

Usage

```
get_test_sets(fs, gh)
```

Arguments

fs	flowSet to gate.
gh	Gating hierarchy.

Value

List of dataframes.

Examples

```
get_test_sets()
```

get_train_data	<i>get_train_data</i>
----------------	-----------------------

Description

function to import training data based on paths to files.

Usage

```
get_train_data(  
  paths_file = NULL,  
  df_paths = NULL,  
  n_cores = 1,  
  prop_down = NULL,  
  remove_class = NULL,  
  n_points_per_plot = NULL,  
  normalize_data = T,  
  vec_col = NULL  
)
```

Arguments

<code>paths_file</code>	Vector of paths. Each path points toward a single csv file containin training info (labels and bivariate expression). <code>paths_file</code> can be also directly the list of dataframes containing labels and bivariate expression.
<code>df_paths</code>	Dataframe containing the paths of file to read. The paths to data must be in the first column. The associated paths to classes are in second column.
<code>n_cores</code>	Number of cores. Default to 1.
<code>prop_down</code>	Proportion of events (downsampling). Default to NULL (downsampling using number of points).
<code>remove_class</code>	Vector of classes to ignore. Default to NULL.
<code>n_points_per_plot</code>	Number of points for downsampling.
<code>normalize_data</code>	If True, data is normalized to 0-1 range. Default to True.
<code>vec_col</code>	vector of columns names if the input dataframes have more than 3 columns. The third column name must always refer to the column with the gate label of each event. Default to NULL.

Value

Dataframe.

Examples

```
get_train_data()
```

```
get_weights_density_features
      get_weights_density_features
```

Description

function to calculate final score based on density features.

Usage

```
get_weights_density_features(df_scores)
```

Arguments

<code>df_scores</code>	Dataframe of distance scores.
------------------------	-------------------------------

Value

Vector of numbers.

Examples

```
get_weights_density_features()
```

import_gating_info	<i>import_sample_gated</i>
--------------------	----------------------------

Description

convert a flowWorkspace or a GatingML file of the train data folder into a gated gh (gh_train)

Usage

```
import_gating_info(path, type = "gs", group_wsp = NULL)
```

Arguments

path	path to gs or GatingML object.
type	type of object.
group_wsp	Group of wsp to import.

Value

GatingSet object

Examples

```
import_sample_gated()
```

import_reference_csv	<i>import_reference_csv</i>
----------------------	-----------------------------

Description

function to import plain gold standards data (no hierarchy)

Usage

```
import_reference_csv(path_results, n_cores = 1)
```

Arguments

path_results	path to directory containing the csv files to read (with third column of labels).
n_cores	Number of cores to use. Default to 1.

Value

list of dataframes

Examples

```
import_reference_csv()
```

```
import_test_set_csv    import_test_set_csv
```

Description

function to import test set in csv format.

Usage

```
import_test_set_csv(path_data, n_cores = 1, xy_col = T)
```

Arguments

path_data	path to directory containing csv files to read (third column is ignored).
n_cores	Number cores. Default to 1.
xy_col	Colnames equal to x and y. Default to True.

Value

List of dataframes.

Examples

```
import_test_set_csv()
```

```
import_test_set_fcs    import_test_set
```

Description

Read the ungated fcs files into a flowSet.

Usage

```
import_test_set_fcs(
  path,
  n_samples = "All",
  ref_f_n = 1,
  pattern_filter = NULL
)
```

Arguments

path	path of directory containig the fcs files.
n_samples	Number of samples. Default to All.
ref_f_n	Set reference flowFrame to match channel names. Default to 1(first flowFrame).
pattern_filter	Filter files to import based on string patterns. Default to NULL.

Value

flowSet.

Examples

```
import_test_set()
```

magicGating

magicGating

Description

function to manually gate samples in a flowSet.

Usage

```
magicGating(
  fs,
  sample_id = 1,
  channel_x,
  channel_y,
  gs_node = NULL,
  label_pol = "1",
  ...
)
```

Arguments

fs	An object of class flowSet. Can be also an object of class cytoset (it will be converted to flowSet) or a GatingSet object (in this case, gs_node is mandatory)
sample_id	Names of the samples to gate. It can also be the numerical index of the sample. Default to sample 1.
channel_x	Name of the channel (x-axis).
channel_y	Name of the channel (y-axis).
gs_node	Name of the node to extract data, if fs is a GatingSet object this a mandatory argument.
label_pol	Label of the gate polygon. Default to "1".

Value

List of two objects of class List. list_poly_gates: List of polygon coordinates. Each element refers to the coordinates of one sample. list_gated_data: List of gated data. Each element refers to the gated data of one sample.

Examples

```
magicGating()
```

magicPlot

magicPlot

Description

function to generate the scatter plot with colored density of the events.

Usage

```
magicPlot(
  df,
  type = "dens",
  polygons_coords_list = NULL,
  show_legend = T,
  size_axis_text = 18,
  size_title_x = 20,
  size_title_y = 20,
  treat_0_as_gate = F,
  x_lab = NULL,
  y_lab = NULL,
  gates_to_plot = NULL,
  apply_manual_scale = F,
  hull_only = F,
  size_points = 1,
  concavity_val = 20,
  aspect_ratio = NULL,
  x_lim1 = NULL,
  x_lim2 = NULL,
  y_lim1 = NULL,
  y_lim2 = NULL,
  add_labels = F,
  map_label_polygon = NULL,
  size_pol_name = 6,
  show_marginals = F,
  ...
)
```

Arguments

df	Dataframe of bivariate markers expression (with labels if gates to plot).
type	Type of plot to generated. "dens"= plot with bivariate density. "ML"= plot with color layer based on machine learning class assignments
polygons_coords_list	list of gates coordinates. Needed if labels not included in df. Default to NULL.
show_legend	Show legend if type="ML". Default to True.
size_axis_text	Size of axis ticks labels. Default to 18.

size_title_x	Size of x axis label title. Default to 20.
size_title_y	Size of y axis label title. Default to 20.
treat_0_as_gate	Treat 0 label as gate. Default to False (0 label is background)
x_lab	Label of x axis. Default to NULL (inherited from input dataframe by default).
y_lab	Label of y axis. Default to NULL (inherited from input dataframe by default).
gates_to_plot	Select labels to plot.
apply_manual_scale	Apply predefined scale of colors. Default to True.
size_points	Size of points in scatter plot.
concavity_val	Concavity value. Default to 20. Higher value, less jagged boundaries
aspect_ratio	Aspect ratio value. If = 1, y and x axis have equal distance between the ticks labels. Default to NULL.
x_lim1	Minimum limit x axis. Default to NULL.
x_lim2	Max limit x axis. Default to NULL.
y_lim1	Minimum limit y axis. Default to NULL.
y_lim2	Max limit y axis. Default to NULL.
add_labels	add polygon labels. Default to FALSE.
map_label_polygon	map of polygon labels (to assign custom labels). Default to NULL.
size_pol_name	size polygon labels. Default to 6.
show_marginals	show 1D density next to axis. Default to False.

Value

Object of class ggplot

Examples

```
magicPlot()
```

magicplot_3D

magicplot_3D

Description

function to make a 3D plot of data.

Usage

```
magicplot_3D(  
  df,  
  class_col = F,  
  x_lab = "x",  
  y_lab = "y",  
  z_lab = "z",  
  type = "ML",  
  size_p = 1  
)
```

Arguments

df	input dataframe composed of at least three columns. Four columns if gate needs to be plot.
class_col	Gate to plot? Default to False.
x_lab	label of the x axis
y_lab	Label of the y axis
z_lab	Label of the z axis
type	Assuming class_col==T, If type==ML, Generate a plot colored based on the gate assignment. If type=="mesh", generate a 3d polygon gate.
size_p	size of scatter plot points. Default to 1.

Value

plotly plot

Examples

```
magicplot_3D()
```

magicPlot_fs	<i>magicPlot_fs</i>
--------------	---------------------

Description

function to plot FCM data directly from objects of class flowSet or flowFrame. The data is shown in a density scatter plot.

Usage

```
magicPlot_fs(fs, sample_id, channel_x, channel_y, ...)
```

Arguments

<code>fs</code>	object of class <code>flowSet</code> or <code>flowFrame</code> .
<code>sample_id</code>	Name of the sample data to plot. No effect if <code>fs</code> is an object of class <code>flowFrame</code> .
<code>channel_x</code>	Name of the channel to plot (x-axis).
<code>channel_y</code>	Name of the channel to plot (y-axis).

Value

Object of class `ggplot`

Examples

```
magicPlot_fs()
```

<code>magicPlot_gs_gates</code>	<i>magicPlot_gs_gates</i>
---------------------------------	---------------------------

Description

Plot one sample from a `GatingSet` with one or more gated populations overlaid.

Usage

```
magicPlot_gs_gates(
  gs,
  sample_id,
  gate_names,
  size_points = 0.5,
  concavity_val = 50,
  ...
)
```

Arguments

<code>gs</code>	A <code>GatingSet</code> object.
<code>sample_id</code>	Character. Name of the sample to plot. Must be present in <code>sampleNames(gs)</code> .
<code>gate_names</code>	Character vector. One or more <code>GatingSet</code> node names to extract and display.
<code>size_points</code>	Numeric. Point size passed to <code>magicPlot()</code> . Default is 0.5.
<code>concavity_val</code>	Numeric. Concavity parameter available for downstream polygon plotting. Default is 50.
<code>...</code>	Additional arguments passed to <code>magicPlot()</code> , such as <code>type</code> , <code>size_axis_text</code> , <code>size_title_x</code> , or <code>size_title_y</code> .

Details

This function extracts one or more gated populations from a single sample in a GatingSet and visualizes them using `magicPlot()`. If multiple gate names are supplied, the extracted gated data are merged so the selected populations are displayed together on the same bivariate plot. Each population is labeled using its node name.

Value

A ggplot object showing the selected sample and gated populations.

Examples

```
# Plot one gate
p1 <- magicPlot_gs_gates(
  gs = gs,
  sample_id = "sample_01.fcs",
  gate_names = "Live_cells",
  size_points = 0.5
)

# Plot multiple gates together
p2 <- magicPlot_gs_gates(
  gs = gs,
  sample_id = "sample_01.fcs",
  gate_names = c("Live_cells", "Dead_cells"),
  type = "ML",
  size_points = 0.5
)
```

magicPlot_gs_hierarchy
magicPlot_gs_hierarchy

Description

Plot all gates from one GatingSet sample as a hierarchy-style figure.

Usage

```
magicPlot_gs_hierarchy(
  gs,
  sample_id,
  n_col_wrap = 4,
  size_points = 0.5,
  size_title = NULL,
  concavity_val = 50,
  add_labels = TRUE,
```



```

    size_pol_name = NULL,
    auto_size = TRUE,
    return_plot_list = FALSE,
    path_output = NULL,
    plot_width = 4,
    plot_height = 4,
    dpi = 300,
    ...
)

```

Arguments

<code>gs</code>	A GatingSet object.
<code>sample_id</code>	Character. Name of the sample to plot. The value must be present in <code>sampleNames(gs)</code> .
<code>n_col_wrap</code>	Integer. Number of columns used to arrange the wrapped hierarchy figure. Default is 4.
<code>size_points</code>	Numeric. Point size passed to <code>magicPlot_gs_gates()</code> and ultimately to <code>magicPlot()</code> . Default is 0.5.
<code>size_title</code>	Numeric or NULL. Size of the title for each subplot. Each subplot title indicates the parent population, for example "Parent pop: CD45+". If NULL and <code>auto_size = TRUE</code> , the value is computed automatically from the grid size. If NULL and <code>auto_size = FALSE</code> , the default value is 10.
<code>concavity_val</code>	Numeric. Concavity value used when polygon boundaries are estimated from gated events. This value is passed to <code>magicPlot_gs_gates()</code> and then to <code>magicPlot()</code> . Larger values generally produce smoother polygon boundaries. Default is 50.
<code>add_labels</code>	Logical. If TRUE, gate names are printed inside the polygon gates. This is passed to <code>magicPlot()</code> . Labels are only shown for plotting modes that support polygon labels, such as <code>type = "dens"</code> . Default is TRUE.
<code>size_pol_name</code>	Numeric or NULL. Size of the gate labels printed inside polygon gates. If NULL and <code>auto_size = TRUE</code> , the value is computed automatically from the grid size. If NULL and <code>auto_size = FALSE</code> , the default value is 4.
<code>auto_size</code>	Logical. If TRUE, automatically estimates text sizes for axis tick labels, axis titles, polygon labels, and subplot titles based on the number of rows and columns in the wrapped figure. User-supplied values passed through <code>...</code> , such as <code>size_axis_text</code> , <code>size_title_x</code> , or <code>size_title_y</code> , are respected and not overwritten. Default is TRUE.
<code>return_plot_list</code>	Logical. If FALSE, return only the wrapped plot. If TRUE, return a list containing the wrapped plot and intermediate objects used to build it. Default is FALSE.
<code>path_output</code>	Character or NULL. Full file path where the wrapped hierarchy plot should be saved. If NULL, no file is exported. The file format is inferred by <code>ggplot2::ggsave()</code> from the file extension, for example <code>.tiff</code> , <code>.png</code> , or <code>.pdf</code> . Default is NULL.
<code>plot_width</code>	Numeric. Width, in inches, assigned to each individual subplot when exporting the full wrapped figure. The final exported width is calculated as <code>plot_width * number_of_columns</code> . Default is 4.

<code>plot_height</code>	Numeric. Height, in inches, assigned to each individual subplot when exporting the full wrapped figure. The final exported height is calculated as <code>plot_height * number_of_rows</code> . Default is 4.
<code>dpi</code>	Numeric. Resolution used when exporting raster image formats such as .png or .tiff. Default is 300.
<code>...</code>	Additional plotting arguments passed to <code>magicPlot_gs_gates()</code> and then to <code>magicPlot()</code> . Common examples include <code>type</code> , <code>size_axis_text</code> , <code>size_title_x</code> , <code>size_title_y</code> , <code>show_legend</code> , <code>x_lim1</code> , <code>x_lim2</code> , <code>y_lim1</code> , and <code>y_lim2</code> .

Details

This function generates a wrapped figure containing one bivariate plot for each gating step in a GatingSet hierarchy. For each subplot, the title indicates the parent population and the gate labels indicate the child population or populations shown on that plot.

Gates that share the same bivariate dimensions are automatically grouped into the same subplot. For example, if NK_cells, B_cells, and T_cells are sibling gates drawn on the same two channels, they are displayed together in one plot. Gates that do not share dimensions with sibling gates are plotted individually.

The hierarchy order is determined from `get_hierarchy_all_pops()`, and the final plots are combined using `patchwork::wrap_plots()`.

The function uses `get_hierarchy_all_pops()` to obtain a hierarchy table. The table is walked row by row so that plots follow the gating hierarchy order. If a child population has an empty Dimensions value, it is plotted alone. If multiple child populations share the same Dimensions value, they are grouped into the same subplot and shown together.

The plotting itself is delegated to `magicPlot_gs_gates()`. This wrapper determines which gates should appear in each subplot, then calls `magicPlot_gs_gates()` repeatedly and combines the resulting plots.

Value

If `return_plot_list = FALSE`, a wrapped patchwork plot object. If `return_plot_list = TRUE`, a list with the following elements:

plot The final wrapped hierarchy plot.

plot_list A named list of individual ggplot objects before wrapping.

plot_groups A named list describing which gate or gates were plotted in each subplot.

plot_titles A named list containing the parent population title for each subplot.

df_tree The hierarchy table returned by `get_hierarchy_all_pops()` and used to build the figure.

Examples

```
# Plot the full hierarchy for one sample
p <- magicPlot_gs_hierarchy(
  gs = gs,
  sample_id = "sample_01.fcs",
  n_col_wrap = 3,
  type = "dens"
```

```

)

# Plot and export the hierarchy figure as a TIFF file
p <- magicPlot_gs_hierarchy(
  gs = gs,
  sample_id = "sample_01.fcs",
  n_col_wrap = 3,
  type = "dens",
  path_output = "~/main/Results/sample_01_hierarchy.tiff",
  plot_width = 4,
  plot_height = 4,
  dpi = 300
)

# Manually control text sizes instead of using automatic sizing
p <- magicPlot_gs_hierarchy(
  gs = gs,
  sample_id = "sample_01.fcs",
  n_col_wrap = 3,
  auto_size = FALSE,
  size_title = 10,
  size_pol_name = 4,
  size_axis_text = 8,
  size_title_x = 10,
  size_title_y = 10,
  type = "dens"
)

```

magicPlot_gs_node

magicPlot_gs_node

Description

Plot selected channels from one GatingSet node.

Usage

```
magicPlot_gs_node(gs_input, node_name, sample_id, channel_x, channel_y, ...)
```

Arguments

<code>gs_input</code>	A GatingSet object.
<code>node_name</code>	Character. Name of the population node to extract, such as "root", "Singlets", or "Live_cells".
<code>sample_id</code>	Sample identifier used to select one sample from the GatingSet. This can be a sample name or an index, depending on <code>get_flowframe_from_gs()</code> .
<code>channel_x</code>	Character. Name of the channel to plot on the x-axis.
<code>channel_y</code>	Character. Name of the channel to plot on the y-axis.
<code>...</code>	Additional arguments passed to <code>magicPlot_fs()</code> .

Details

This function extracts one sample and one population node from a GatingSet as a flowFrame, then plots two selected channels using magicPlot_fs(). Unlike magic_plot_gs_gates(), this function does not overlay multiple gate labels. It simply plots the events contained in the selected node.

Value

A plot object returned by magicPlot_fs().

Examples

```
magicPlot_gs_node(
  gs_input = gs,
  node_name = "Live_cells",
  sample_id = "sample_01.fcs",
  channel_x = "FSC-A",
  channel_y = "LIVE DEAD Blue-A"
)
```

magicPlot_template	<i>magicPlot_template</i>
--------------------	---------------------------

Description

function to interactively generate a template based on bivariate ungated data.

Usage

```
magicPlot_template(df, size_points = 1)
```

Arguments

df	input dataframe composed of two columns reporting markers expression (first column = marker 1, second column = marker 2).
size_points	Size of points in scatter plot. Default to 1.

Value

Dataframe

Examples

```
magicPlot_template()
```

magicPred

magicPred

Description

function to predict on plain test data (no hierarchy)

Usage

```
magicPred(
  test_data,
  magic_model = NULL,
  magic_model_n_gates = NULL,
  ref_model_info = NULL,
  n_cores = 1,
  ref_data_train = NULL,
  prop_down = NULL,
  thr_dist = 0.05,
  n_points_per_plot = NULL,
  normalize_data = T,
  include_zero_val = T,
  ...
)
```

Arguments

<code>test_data</code>	Dataframe of test data to gate. It has only the two columns of marker expression.
<code>magic_model</code>	Global trained model to predict gates. It can be a single model or list of named models (each model trained on selected number of gates).
<code>magic_model_n_gates</code>	Global trained model to predict number of gates. If different from <code>NULL</code> , <code>magic_model</code> is expected to be a list of models to predict certain gates (e.g., 5 models for 2,3,4,5 or 6 gates).
<code>ref_model_info</code>	Template model to predic gates.
<code>n_cores</code>	Number of cores to use. Default to 1.
<code>ref_data_train</code>	Template data used to generate <code>ref_model_info</code> . Needed to calculate target-template distance.
<code>prop_down</code>	Proportion for downsampling. Default to <code>NULL</code> (automatic downsampling using <code>n_points_per_plot</code>).
<code>n_points_per_plot</code>	Number of points to consider for downsampling. Default to 500.
<code>normalize_data</code>	If True, data is normalized to 0-1 range. Default to True.
<code>include_zero_val</code>	considering events labeled as 0 as an additional gate when there is only one gate. Default to True.

Value

List of Dataframes.

Examples

magicPred()

magicPred_all	<i>magicPred_all</i>
---------------	----------------------

Description

function to predict on plain test data (no hierarchy)

Usage

```
magicPred_all(  
  list_test_data,  
  sample_id = NULL,  
  magic_model = NULL,  
  ref_model_info = NULL,  
  magic_model_n_gates = NULL,  
  ref_data_train = NULL,  
  prop_down = NULL,  
  n_points_per_plot = NULL,  
  n_gates_df = NULL,  
  thr_dist = 0.05,  
  n_cores = 1,  
  normalize_data = T,  
  include_zero_val = T,  
  n_cores_all = 1,  
  verbose = F,  
  ...  
)
```

Arguments

- list_test_data List of unlabeled dataframes. It has only the two columns of marker expression.
- sample_id Id of the samples to gate. Could be a vector containing the names or numerical indices (position) of the samples to gate. If NULL, all samples in list_test_data are gated. Default to NULL.
- magic_model Global trained model to predict gates. It can be a single model or list of named models (each model trained on selected number of gates).
- ref_model_info Template model to predic gates.

magic_model_n_gates	Global trained model to predict number of gates. If different from NULL, magic_model is expected to be a list of models to predict certain gates (e.g., 5 models for 2,3,4,5 or 6 gates).
ref_data_train	Template data used to generate ref_model_info. Needed to calculate target-template distance.
n_points_per_plot	Number of points to consider for downsampling. Default to 500. Note: the gates in the downsampled data are projected back to the original data to get the actual populations.
n_gates_df	Dataframe pairing the sample name (first column) with the user-defined number of gates (second column). Default to NULL.
thr_dist	Distance threshold for centroids calculation during post processing. Default to 0.15.
n_cores	Number of cores to use to process one sample. Default to 1.
normalize_data	If True, data is normalized to 0-1 range. Default to True.
include_zero_val	considering events labeled as 0 as an additional gate when there is only one gate. Default to True.
n_cores_all	Number of cores to use across all samples. Default to 1.
verbose	If True, print all message and disable tryCatch (any error will stop the execution). Default to False.

Value

List of Dataframes.

Examples

```
magicPred_all()
```

magicPred_hierarchy	<i>magicPred_hierarchy</i>
---------------------	----------------------------

Description

function to predict the gates on the ungated .fcs samples.

Usage

```
magicPred_hierarchy(list_test_sets, list_models_local, df_tree, n_cores = 1)
```

Arguments

`list_test_sets` contains the list of root dataframe for each ungated fcs file imported.

`list_models_local` contains the optimized local models pre-generated using the `magicTrain_local` function.

`df_tree` contains the info related to the populations hierarchy.

`n_cores` Number of cores to use. Default to 1.

Value

List of Dataframes.

Examples

```
magicPred_hierarchy()
```

magicTrain

magicTrain

Description

function to generate one training model based on a list of training sets (no hierarchy).

Usage

```
magicTrain(
  df_train,
  n_cores = 1,
  train_model = "rf",
  k_cv = 10,
  list_index_train = NULL,
  list_index_val = NULL,
  n_tree = 10,
  tune_lenght = 3,
  size_nnet_units = 100,
  decay_nnet = 0.1,
  method_control = "auto",
  type_y = "classes",
  seed_n = 40
)
```


Arguments

df_train	training dataframe generated by the get_train_data function.
n_cores	Number of cores to use. Default to 1.
train_model	Type of training model. Default to rf.
k_cv	Number of k for cross-validation (if method control=cv)
list_index_train	List of vector of indices to use in training for each fold.
list_index_val	List of vector of indices to use as held out data for each fold.
n_tree	Number of trees for random forest.
tune_lenght	Number of parameters values trained during cross-validation.
size_nnet_units	Number of units in hidden layer (if train_model=nnet).
decay_nnet	Decay parameter value for nnet model.
method_control	Type of training control: oob or cv or auto. Default to auto.
type_y	Type of response variable: classes (train to predict gates boundaries) or n_gates_info(train to predict number of gates).
seed_n	Set seed. Default to 40.

Value

model object.

Examples

```
magicTrain()
```

```
magicTrain_dt
```

```
magicTrain_dt
```

Description

function to generate a random forest training model.

Usage

```
magicTrain_dt(
  Xtrain,
  Ytrain,
  k_cv = 10,
  list_index_train = NULL,
  list_index_val = NULL,
  tune_lenght = 5
)
```

Arguments

Xtrain	Dataframe of training features.
Ytrain	Dataframe of labels (one column).
k_cv	Number of k for cross-validation.
list_index_train	List of vector of indices to use in training for each fold.
list_index_val	List of vector of indices to use as held out data for each fold.
tune_lenght	Number of hyper parameters to test. Default to 5.

Value

model object.

Examples

```
magicTrain_dt()
```

magicTrain_hierarchy *magicTrain_local*

Description

function to generate the local training models using the list of hierarchical training set.

Usage

```
magicTrain_hierarchy(  
  list_train_sets,  
  n_tree = 10,  
  train_model = "rf",  
  method_control = "oob",  
  n_cores = 1  
)
```

Arguments

list_train_sets	List of labeled dataframe to train generated by the get_local_train function.
n_tree	Number of tree for random forest model.
train_model	Type of training model. Default to random forest ("rf").
method_control	Cross-validation method. Default to out-of-the-bag method (oob).
n_cores	Number of cores to use. Default to 1.

Value

List of models objects.

Examples

```
magicTrain_local()
```

magicTrain_knn	<i>magicTrain_knn</i>
----------------	-----------------------

Description

function to generate a random forest training model.

Usage

```
magicTrain_knn(
  Xtrain,
  Ytrain,
  k_cv = 10,
  list_index_train = NULL,
  list_index_val = NULL,
  tune_lenght = 5
)
```

Arguments

Xtrain	Dataframe of training features.
Ytrain	Dataframe of labels (one column).
k_cv	Number of k for cross-validation.
list_index_train	List of vector of indices to use in training for each fold.
list_index_val	List of vector of indices to use as held out data for each fold.
tune_lenght	Number of hyper parameters to test. Default to 5.

Value

model object.

Examples

```
magicTrain_knn()
```

magicTrain_nb	<i>magicTrain_nb</i>
---------------	----------------------

Description

function to generate a random forest training model.

Usage

```
magicTrain_nb(  
  Xtrain,  
  Ytrain,  
  k_cv = 10,  
  list_index_train = NULL,  
  list_index_val = NULL,  
  tune_lenght = 5  
)
```

Arguments

Xtrain	Dataframe of training features.
Ytrain	Dataframe of labels (one column).
k_cv	Number of k for cross-validation.
list_index_train	List of vector of indices to use in training for each fold.
list_index_val	List of vector of indices to use as held out data for each fold.
tune_lenght	Number of hyper parameters to test. Default to 5.

Value

model object.

Examples

```
magicTrain_nb()
```

magicTrain_nnet	<i>magicTrain_nnet</i>
-----------------	------------------------

Description

function to generate a neural net training model.

Usage

```
magicTrain_nnet(  
  Xtrain,  
  Ytrain,  
  k_cv = 10,  
  list_index_train = NULL,  
  list_index_val = NULL,  
  size = 100,  
  decay = 0.1,  
  tune_lenght = 5  
)
```

Arguments

Xtrain	Dataframe of training features.
Ytrain	Dataframe of labels (one column).
k_cv	Number of k for cross-validation.
list_index_train	List of vector of indices to use in training for each fold.
list_index_val	List of vector of indices to use as held out data for each fold.
size	Number of units in hidden layer (if train_model=nnet).
decay	Decay parameter value for nnet model.

Value

model object.

Examples

```
magicTrain_nnet()
```

magicTrain_rf	<i>magicTrain_rf</i>
---------------	----------------------

Description

function to generate a random forest training model.

Usage

```
magicTrain_rf(  
  Xtrain,  
  Ytrain,  
  list_index_train = NULL,  
  list_index_val = NULL,  
  n_tree = 10,  
  method_control = "oob",  
  k_cv = 10  
)
```

Arguments

Xtrain	Dataframe of training features.
Ytrain	Dataframe of labels (one column).
list_index_train	List of vector of indices to use in training for each fold.
list_index_val	List of vector of indices to use as held out data for each fold.
n_tree	Number of trees for random forest.
method_control	Type of training control: oob or cv. Default to oob.
k_cv	Number of k for cross-validation (if method control=cv).

Value

model object.

Examples

```
magicTrain_rf()
```

<code>magic_label_poly</code>	<i>magic_label_poly</i>
-------------------------------	-------------------------

Description

function to label points of dataframe based on polygon coordinates

Usage

```
magic_label_poly(df, polygon_df, label_pol = "1")
```

Arguments

- | | |
|-------------------------|---|
| <code>df</code> | Dataframe composed of two columns for marker expression of first (x axis = first column) and second marker (y axis = second column) |
| <code>polygon_df</code> | Dataframe containing the x coordinates (first column) and y coordinates (second column) of the current polygon to label. |
| <code>label_pol</code> | Label for current polygon. |

Value

Dataframe

Examples

```
magic_label_poly()
```

<code>magic_label_rectangle</code>	<i>magic_label_rectangle</i>
------------------------------------	------------------------------

Description

function to label points of dataframe based on rectangle coordinates

Usage

```
magic_label_rectangle(df, x_min, x_max, y_min, y_max, label_pol = "1")
```

Arguments

- | | |
|------------------------|---|
| <code>df</code> | Dataframe composed of two columns for marker expression of first (x axis = first column) and second marker (y axis = second column) |
| <code>x_min</code> | x coordinate minimum |
| <code>x_max</code> | x coordinate maximum |
| <code>y_min</code> | y coordinate minimum |
| <code>y_max</code> | y coordinate maximum |
| <code>label_pol</code> | Label for current polygon. |

Value

Dataframe

Examples

```
magic_label_rectangle()
```

```
magic_manual_gating    magic_manual_gating
```

Description

Perform interactive manual gating on selected samples and add or update the resulting polygon gate in a GatingSet.

Usage

```
magic_manual_gating(
  gs_input,
  parent_node,
  new_child_node,
  samples_id,
  channel_x,
  channel_y,
  size_points = 0.5,
  return_list_out = F,
  input_list_out = NULL,
  return_gs_only = T,
  ...
)
```

Arguments

<code>gs_input</code>	A GatingSet object.
<code>parent_node</code>	Character. Name of the parent node where the new gate should be added or updated.
<code>new_child_node</code>	Character. Name of the child node to create or update.
<code>samples_id</code>	Character vector. Sample names to manually gate. These names must match <code>sampleNames(gs_input)</code> .
<code>channel_x</code>	Character. Name of the channel to plot on the x-axis.
<code>channel_y</code>	Character. Name of the channel to plot on the y-axis.
<code>size_points</code>	Numeric. Point size used in the interactive gating plot. Default is 0.5.
<code>return_list_out</code>	Logical. If TRUE, return only the output from <code>magicGating()</code> without modifying the GatingSet. Default is FALSE.

- `input_list_out` Optional list. Existing output from `magicGating()` containing a `list_poly_gates` slot. If provided, interactive gating is skipped and these polygon gates are added or updated in the `GatingSet`.
- `return_gs_only` Logical. If `TRUE`, return only the modified `GatingSet`. If `FALSE`, return a list containing the modified `GatingSet` and the gating output. Default is `TRUE`.
- `...` Additional arguments passed to `magicGating()`.

Details

This function wraps `magicGating()` for one manual polygon gate and integrates the resulting `polygonGate` into a `GatingSet`. If the target child node does not already exist, it is created under `parent_node`. If the node already exists, the gate is updated only for the samples present in the gating output.

Value

If `return_gs_only = TRUE`, a modified `GatingSet`. Otherwise, a list with two elements: `gs`, the modified `GatingSet`, and `list_out`, the gating output used to add or update the gate.

Examples

```
gs <- magic_manual_gating(  
  gs_input = gs,  
  parent_node = "root",  
  new_child_node = "Live_cells",  
  samples_id = c("sample_01.fcs"),  
  channel_x = "FSC-A",  
  channel_y = "LIVE DEAD Blue-A"  
)
```

<code>magic_plot_wrap</code>	<i>magic_plot_wrap</i>
------------------------------	------------------------

Description

function to wrap all plots in one plot

Usage

```
magic_plot_wrap(list_gated_data, n_col_wrap = 3, size_title = 10, ...)
```

Arguments

- `list_gated_data` List of dataframes composed by three columns (could be generated by the `get_list_df_gated_plots()` function)
- `n_col_wrap` number of columns in the wrapped plot. Default to 3.
- `size_title` size of title of each plot. Default to 10.

Value

wrapped plot

Examples

```
magic_plot_wrap()
```

```
magic_template_predict
      magic_template_predict
```

Description

Predict gated populations from a trained flowMagic template model and optionally add the predicted gates to a GatingSet.

Usage

```
magic_template_predict(
  gs_input,
  parent_node,
  channel_x,
  channel_y,
  ref_train,
  ref_model_info,
  label_to_node,
  n_cores = 1,
  concavity_val = 50,
  add_to_gs = TRUE,
  return_gs_only = FALSE,
  ...
)
```

Arguments

gs_input	A GatingSet object.
parent_node	Character. Name of the parent node from which raw data are exported and under which predicted gates are added.
channel_x	Character. Name of the channel to use on the x-axis.
channel_y	Character. Name of the channel to use on the y-axis.
ref_train	Data frame. Training data generated by get_train_data(), typically train_out\$ref_train from magic_template_train().
ref_model_info	List. Trained model information returned by magicTrain(), typically train_out\$ref_model_info from magic_template_train().

label_to_node	Named character vector. Names are predicted labels and values are the corresponding GatingSet node names. For example, <code>c("1" = "Live_cells", "2" = "Dead_cells")</code> .
n_cores	Integer. Number of cores used for prediction and polygon extraction. Default is 1.
concavity_val	Numeric. Concavity parameter passed to <code>flowmagic_pred_to_poly_gates()</code> . Higher values generally produce smoother polygon gates. Default is 50.
add_to_gs	Logical. If TRUE, convert predicted labels to polygon gates and add or update them in the GatingSet. If FALSE, only prediction results are returned. Default is TRUE.
return_gs_only	Logical. If TRUE, return only the modified GatingSet. Default is FALSE.
...	Additional arguments passed to <code>export_raw_gs_plots()</code> .

Details

This function exports bivariate expression data from a selected GatingSet node, predicts event labels using a previously trained flowMagic model, converts selected predicted labels into polygon gates, and optionally adds or updates these gates in the input GatingSet.

The `label_to_node` argument controls how predicted labels are mapped to GatingSet node names. For example, `c("1" = "Live_cells", "2" = "Dead_cells")` converts predicted label "1" into a node named "Live_cells" and predicted label "2" into a node named "Dead_cells".

Value

If `return_gs_only = TRUE`, a modified GatingSet. Otherwise, a list with four elements: `gs`, the modified or original GatingSet; `list_test_data`, the exported test data; `list_dfs_pred`, the predicted labeled data frames; and `list_poly_gates`, the polygon gates generated for each requested node.

Examples

```
pred_out <- magic_template_predict(
  gs_input = gs,
  parent_node = "root",
  channel_x = "FSC-A",
  channel_y = "LIVE DEAD Blue-A",
  ref_train = train_out$ref_train,
  ref_model_info = train_out$ref_model_info,
  label_to_node = c("1" = "Live_cells", "2" = "Dead_cells"),
  n_cores = 1,
  add_to_gs = TRUE
)

gs <- pred_out$gs
```

magic_template_train *magic_template_train*

Description

Build a template training set from manually gated samples and train a flowMagic classification model.

Usage

```
magic_template_train(
  gs_input,
  parent_node,
  samples_id,
  channel_x,
  channel_y,
  size_points = 0.5,
  n_gates = 1,
  input_list_out_final = NULL,
  train_model = "rf",
  ...
)
```

Arguments

<code>gs_input</code>	A GatingSet object.
<code>parent_node</code>	Character. Name of the parent node from which expression data are extracted for template gating.
<code>samples_id</code>	Character vector. Sample names to manually gate as template samples.
<code>channel_x</code>	Character. Name of the channel to plot on the x-axis.
<code>channel_y</code>	Character. Name of the channel to plot on the y-axis.
<code>size_points</code>	Numeric. Point size used in the interactive gating plot. Default is 0.5.
<code>n_gates</code>	Integer. Number of template gates to draw per selected sample. Each gate is assigned a label from 1 to <code>n_gates</code> . Default is 1.
<code>input_list_out_final</code>	Optional list. Existing template data object, usually from a previous call to <code>magic_template_train()</code> <code>\$list_out_final</code> . If supplied, newly gated templates are added to this existing template set. Samples with matching names are replaced.
<code>train_model</code>	Character. Model type passed to <code>magicTrain()</code> . Default is "rf".
<code>...</code>	Additional arguments passed to <code>magicGating()</code> .

Details

This function performs one or more rounds of interactive manual gating on selected samples. Each gate is assigned a different label. If more than one gate is drawn, the labeled outputs are merged into a single template data object. The resulting templates are converted into training data using `get_train_data()` and used to train a model with `magicTrain()`.

Existing template data can be supplied through `input_list_out_final`, allowing users to add new manually gated templates without redrawing previous ones.

Value

A list with five elements: `ref_model_info`, the trained model information returned by `magicTrain()`; `ref_train`, the training data returned by `get_train_data()`; `list_out_final`, the complete merged template set; `list_out_new`, the newly generated template data; and `all_list_out_obj`, the raw outputs from each call to `magicGating()`.

Examples

```
train_out <- magic_template_train(
  gs_input = gs,
  parent_node = "root",
  samples_id = c("sample_01.fcs", "sample_02.fcs"),
  channel_x = "FSC-A",
  channel_y = "LIVE DEAD Blue-A",
  n_gates = 2,
  train_model = "rf"
)

# Add additional template samples later
train_out2 <- magic_template_train(
  gs_input = gs,
  parent_node = "root",
  samples_id = c("sample_03.fcs"),
  channel_x = "FSC-A",
  channel_y = "LIVE DEAD Blue-A",
  n_gates = 2,
  input_list_out_final = train_out$list_out_final
)
```

map_to_parent

map_to_parent

Description

function to get binary dataset with gate assignation based on the mother population of the selected pop (instead of the root).

Usage

```
map_to_parent(gh, binary_df)
```

Arguments

gh	GatingHierarchy.
binary_df	Dataframe generated by the map_to_root function.

Value

Dataframe.

Examples

```
map_to_parent()
```

map_to_root

map_to_root

Description

it generates a dataset indicating what cells belong to the selected pop (1) 0 otherwise. Old name: pre_process_manual_binary() The dataset is always the dataset of the Root pop. gh and pop are mandatory arguments, dim and mode have a default value.

Usage

```
map_to_root(gh, pop)
```

Arguments

gh	GatingHierarchy.
pop	Name of the population to get events assignments.

Value

Dataframe.

Examples

```
map_to_root()
```

```
merge_magicGating_labels
      merge_magicGating_labels
```

Description

function to merge list of gated data when gating multiple gates.

Usage

```
merge_magicGating_labels(list_out_1, list_out_2, gated_data_only = F)
```

Arguments

`list_out_1` List of Dataframes to update.
`list_out_2` List of Dataframes to merge with `list_out_1` dataframes.
`gated_data_only` Input contains only gated data, not polygons. Default to False.

Value

list of Dataframes

Examples

```
merge_magicGating_labels()
```

```
name_pop_gating      name_pop_gating
```

Description

function to get the name of all the pops of the gating hierarchy.

Usage

```
name_pop_gating(gh)
```

Arguments

`gh` GatingHierarchy.

Value

Vector of characters.

Examples

```
name_pop_gating()
```

post_process_gates	<i>post_process_gates</i>
--------------------	---------------------------

Description

function to post process the events after model prediction.

Usage

```
post_process_gates(  
  gated_df,  
  n_cores = 1,  
  thr_dist = 0.15,  
  include_zero = F,  
  remove_centroids = T,  
  type = "dist",  
  concavity_val = 5,  
  normalize_data = T,  
  ...  
)
```

Arguments

- `gated_df` dataframe with labels (third column).
- `n_cores` Number of cores. Default to 1.
- `thr_dist` Distance threshold for centroids calculation. Default to 0.15.
- `include_zero` Consider centroid of label 0. Default to False.
- `remove_centroids` Remove centroids too near each other based on `thr_dist` value.
- `type` Type of post-processing.
- `concavity_val` Concavity of polygons for the "polygon" type of post-processing

Value

Dataframe.

Examples

```
post_process_gates()
```

process_test_data	<i>process_test_data</i>
-------------------	--------------------------

Description

function to get test data correctly formatted.

Usage

```
process_test_data(  
  test_data,  
  prop_down = NULL,  
  n_points_per_plot = 500,  
  normalize_data = T  
)
```

Arguments

- test_data Dataframe of bivariate markers expression.
- prop_down Proportion of events (downsampling). Default to NULL (downsampling using number of points).
- n_points_per_plot Number of points for downsampling.
- normalize_data If True, data is normalized to 0-1 range. Default to True.

Value

Dataframe.

Examples

```
process_test_data()
```

range01	<i>range01</i>
---------	----------------

Description

function to put data in range 0-1.

Usage

```
range01(x)
```

Arguments

x Vector of numbers to scale.

Value

Vector of numbers.

Examples

range01()

smooth_hull	<i>smooth_hull</i>
-------------	--------------------

Description

function to smooth concave polygons

Usage

smooth_hull(hull_df, spar = 0.7)

Arguments

hull_df	Dataframe generate by concaveman functions inside get_huget_hull_all_gates function
spar	Spar value to regulate smoothing process: higher value (max 1) higher smoothing.
buffer_dist	Regulate buffer distance before smoothing. Default to 500.

Value

Dataframe.

Examples

smooth_hull()

```
update_label_association  
    update_label_association
```

Description

function to update label association.

Usage

```
update_label_association(df)
```

Arguments

df Dataframe.

Value

Vector.

Examples

```
update_label_association()
```

Index

- * **GatingHierarchy**
 - get_flowframe_from_gs, 16
- * **GatingSet**
 - get_flowframe_from_gs, 16
 - magic_template_predict, 50
 - magicPlot_gs_gates, 31
 - magicPlot_gs_hierarchy, 32
 - magicPlot_gs_node, 35
- * **automated**
 - magic_template_predict, 50
- * **flowFrame**
 - get_flowframe_from_gs, 16
- * **flowMagic**
 - add_labels_column, 3
 - assign_events_to_nearest_centroids, 4
 - check_polygons_intersection, 5
 - compute_gates, 5
 - convert_to_integers_chr, 6
 - csv_to_dens, 6
 - export_raw_gs_plots, 8
 - exports_plots, 7
 - extract_polygon_gates, 9
 - flowmagic_pred_to_gs, 10
 - flowmagic_pred_to_poly_gates, 10
 - get_centroids, 12
 - get_classes_expr_df, 13
 - get_density_features, 14
 - get_density_scores, 14
 - get_dist_template, 15
 - get_distance_loc_vs_test, 15
 - get_flowframe_from_gs, 16
 - get_hierarchy_all_pops, 18
 - get_hull_all_gates, 18
 - get_indices_cross_val, 19
 - get_list_df_gated_plots, 20
 - get_local_train_sets, 20
 - get_pop_multiclass, 22
 - get_pops_hierarchy_list, 21
 - get_slot_hierarchy_list, 22
 - get_test_sets, 23
 - get_train_data, 23
 - get_weights_density_features, 24
 - import_gating_info, 25
 - import_reference_csv, 25
 - import_test_set_csv, 26
 - import_test_set_fcs, 26
 - magic_label_poly, 47
 - magic_label_rectangle, 47
 - magic_manual_gating, 48
 - magic_plot_wrap, 49
 - magic_template_predict, 50
 - magic_template_train, 52
 - magicGating, 27
 - magicPlot, 28
 - magicplot_3D, 29
 - magicPlot_fs, 30
 - magicPlot_gs_gates, 31
 - magicPlot_gs_hierarchy, 32
 - magicPlot_gs_node, 35
 - magicPlot_template, 36
 - magicPred, 37
 - magicPred_all, 38
 - magicPred_hierarchy, 39
 - magicTrain, 40
 - magicTrain_dt, 41
 - magicTrain_hierarchy, 42
 - magicTrain_knn, 43
 - magicTrain_nb, 44
 - magicTrain_nnet, 45
 - magicTrain_rf, 46
 - map_to_parent, 53
 - map_to_root, 54
 - merge_magicGating_labels, 55
 - name_pop_gating, 55
 - post_process_gates, 56
 - process_test_data, 57
 - range01, 57

- smooth_hull, 58
- update_label_association, 59
- * **gating**
 - magic_template_predict, 50
 - magic_template_train, 52
- * **hierarchy**
 - magicPlot_gs_hierarchy, 32
- * **machine-learning**
 - magic_template_train, 52
- * **plotting**
 - magicPlot_gs_gates, 31
 - magicPlot_gs_hierarchy, 32
 - magicPlot_gs_node, 35
- * **prediction**
 - magic_template_predict, 50
- * **template**
 - magic_template_predict, 50
 - magic_template_train, 52
- * **training**
 - magic_template_train, 52
- add_labels_column, 3
- assign_events_to_nearest_centroids, 4
- check_polygons_intersection, 5
- compute_gates, 5
- convert_to_integers_chr, 6
- csv_to_dens, 6
- export_raw_gs_plots, 8
- exports_plots, 7
- extract_polygon_gates, 9
- flowmagic_pred_to_gs, 10
- flowmagic_pred_to_poly_gates, 10
- format_channel_info, 11
- get_centroids, 12
- get_classes_expr_df, 13
- get_density_features, 14
- get_density_scores, 14
- get_dist_template, 15
- get_distance_loc_vs_test, 15
- get_flowframe_from_gs, 16
- get_gate_channels, 17
- get_hierarchy_all_pops, 18
- get_hull_all_gates, 18
- get_indices_cross_val, 19
- get_list_df_gated_plots, 20
- get_local_train_sets, 20
- get_pop_multiclass, 22
- get_pops_hierarchy_list, 21
- get_slot_hierarchy_list, 22
- get_test_sets, 23
- get_train_data, 23
- get_weights_density_features, 24
- import_gating_info, 25
- import_reference_csv, 25
- import_test_set_csv, 26
- import_test_set_fcs, 26
- magic_label_poly, 47
- magic_label_rectangle, 47
- magic_manual_gating, 48
- magic_plot_wrap, 49
- magic_template_predict, 50
- magic_template_train, 52
- magicGating, 27
- magicPlot, 28
- magicplot_3D, 29
- magicPlot_fs, 30
- magicPlot_gs_gates, 31
- magicPlot_gs_hierarchy, 32
- magicPlot_gs_node, 35
- magicPlot_template, 36
- magicPred, 37
- magicPred_all, 38
- magicPred_hierarchy, 39
- magicTrain, 40
- magicTrain_dt, 41
- magicTrain_hierarchy, 42
- magicTrain_knn, 43
- magicTrain_nb, 44
- magicTrain_nnet, 45
- magicTrain_rf, 46
- map_to_parent, 53
- map_to_root, 54
- merge_magicGating_labels, 55
- name_pop_gating, 55
- post_process_gates, 56
- process_test_data, 57
- range01, 57
- smooth_hull, 58
- update_label_association, 59